

Desarrollo de sistemas con tecnología Java - Módulo 3 Principios y Patrones de diseño

Ejercicio 1.1 – Estructura Sistema de gestión de ordenes

Solicitado por: Alfonso Hernandez Xochipa

Objetivo: Se requiere programar un nucleo de procesamiento de ordenes de un e-commerce: desde la creación hasta la posible cancelación o devolución, con distintas reglas según el tipo de pedido.

Instrucciones: (7)

1. Crearemos primero que todo una interfaz que servirá como contrato de diferentes procesadores de ordenes, la interfaz se llamará **OrderProcessor** y tendrá los siguientes métodos definidos.
 - a. processOrder(): boolean – Ejecutará el flujo de procesamiento
 - b. cancelOrder(): boolean – Permitirá cancelar la orden antes del envío.
 - c. generateInvoice(): Invoice – Permitirá crear una nueva factura.
2. Necesitaremos crear una clase **abstracta** llamada **Order** la cual tendrá los siguientes atributos y metodos
 - a. Atributos (*protected*)
 - i. orderId:String
 - ii. orderDate:Date
 - iii. baseAmount:Double
 - b. Constructor
 - i. Public Order() – Constructor por defecto
 - ii. Protected Order(String orderId, Double baseAmount) : Constructor que inicializa orderId y baseAmount, ademas tambien inicializa orderDate.
 - c. Metodos(*protected*)
 - i. logStatus(String status):void – Método concreto que imprimirá un mensaje en consola con orderId y status.
 - ii. **abstract** calculateTotal(): Double – Metodo abstracto que calculará el total de la orden y se aplicarán descuentos según su implementación.
3. Crea dos clases que hereden de *Order* y agregá lógica en cada subclase según el criterio de cada quien.
 - a. StandardOrder – Clase que representa una clase que **es una tipo de Order**
 - i. Agregar lógica en el método calculateTotal() para que agregué un cargo de \$5.0 al baseAmount.
 - b. ExpressOrder - Clase que representa una clase que **es una tipo de Order**

- i. Agregar lógica en el método `calculateTotal()` para que agregué un cargo de \$10.0 al `baseAmount`
4. Crea dos clases que implementen de *OrderProcessor* y sobrescribe sus métodos correspondientes.
 - a. Clases Nuevas
 - i. `StandardOrderProcessor` extends ...
 - ii. `ExpressOrderProcessor` extends ...
 - b. Constructores
 - i. Crea un constructor para cada clase que reciba como parametro un objeto de *Order* e inicializalo en la clase (recuerda crear un atributo de clase encapsulado).
 - c. Lógica de los métodos
 - i. En el método `processOrder()`:
 1. Llamaremos primero a `calculateTotal()`;
 2. Invocaremos `logStatus()` e imprimiremos "Processed: total = " + total
 3. Retornaremos true;
 - ii. En el método `cancelOrder()`:
 1. Imprimiremos el valor del atributo `orderDate` de nuestro objeto *Order*.
 2. Invocaremos `logStatus()` e imprimiremos "Cancelled".
 - iii. En el método `generateInvoice()`:
 1. Crearemos una clase sencilla llamada *Invoice*, agregamos los atributos (`orderId`, `date`, `total`) y retornaremos una nueva instancia de esa clase.
5. Prueba tu implementación!, como ya has programado el código nucleo, ahora viene la clase para probar tu código.
 - a. Crea una clase llamada *OrderApp* y sobrescribe el método `main` para ejecutar tu implementación.
 - b. Necesitarás crear instancias de las clases anteriormente creadas.

NOTA:

- No es necesario agregar logica completa de momento, concéntrate en crear las estructuras como se indica y reforzar los conceptos básicos de OOP.
- Organiza las clases en Packages para mejorar la legibilidad de tu proyecto.

Valor 1 puntos